Completing jupyter notebook TODO sections

> *File: c_sgd_momentum_analysis.ipynb - Colab.pdf*
>
> Please fill in the #TODO sections in this jupyter notebook.

12/9/2025, 9:31:35 AM

I'll help you fill in the TODO sections in this Jupyter notebook about gradient descent with momentum.

There are two TODO sections to complete:

1. First TODO: Implementing Gradient Descent with Momentum

In the `runGDM` function, you need to implement the momentum update rule:

```
##########################################################################
# TODO: implement this method
##########################################################################
smoothed_grad = beta * smoothed_grad + (1 - beta) * grad
##########################################################################
  Claude provides the correct code here for the first TODO
```

This implements exponential moving average of gradients, where `beta` controls how much of the previous smoothed gradient to retain.

2. Second TODO: Experimenting with Different Learning Rates

For Question 2, you need to re-run one of the methods with a different learning rate. Based on the hint that momentum typically allows for larger learning rates, here's what you should add:

```
##########################################################################
# TODO: implement this method
##########################################################################
# Re-run gradient descent with momentum using a larger learning
rate
maxiter = 100
stepsize_new = 5e-4  # Increased from 1e-4 to 5e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_new, beta)

# Keep the original GD results for comparison
# grads, ws, losses from earlier remain unchanged
##########################################################################
```

Claude provides syntactically correct code here for the second TODO, but I opt not to use this implementation and resend the notebook after running through the first TODO

Explanation:

1. **Momentum update**: The smoothed gradient is computed as a weighted average of the previous smoothed gradient and the current gradient. This helps smooth out oscillations and accelerate convergence in consistent directions.

2. **Learning rate experiment**: Momentum allows us to use larger learning rates because it dampens oscillations. By increasing the stepsize from `1e-4` to `5e-4` (5x larger) for the momentum method while keeping vanilla GD at `1e-4`, you should observe:

   - Momentum converges significantly faster
   - The gap between methods is more pronounced in one dimension (likely dimension 0, which has larger eigenvalue)
   - Loss decreases more rapidly for momentum

The key insight is that momentum helps navigate poorly-conditioned loss landscapes where different dimensions have vastly different curvatures (eigenvalues).

12/9/2025, 9:31:48 AM

Claude provides good explanations here for the steps that its taking, the all seem reasonable and of sound judgement.

File: q_sgd_momentum_analysis.ipynb_q1_filled.pdf

I ran all of the cells with your code up to question 2. How would you fill in the second TODO with this new information?

12/9/2025, 9:38:42 AM

Looking at your results, I can see that momentum converges slightly faster than plain GD (the blue line is consistently below the red line in the loss plot). For Question 2, you need to increase the learning rate for the momentum method to show an even more dramatic difference.

Here's how to fill in the second TODO:

I give Claude a second chance here to fill in the second TODO, but it opts to keep the code it filled in previously.

```
##################################################################
# TODO: implement this method
##################################################################
# Re-run gradient descent with momentum using a larger learning
rate
maxiter = 100
stepsize_new = 5e-4  # Increased from 1e-4 to 5e-4 (5x larger)
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_new, beta)

# Keep the plain GD at the original learning rate for comparison
# (grads, ws, losses already exist from the earlier cell)
##################################################################
```

What this does:

- Keeps plain GD at stepsize `1e-4` (from earlier)
- Runs momentum GD at stepsize `5e-4` (5x larger)
- This will show momentum's key advantage: it can handle larger learning rates because it smooths out oscillations

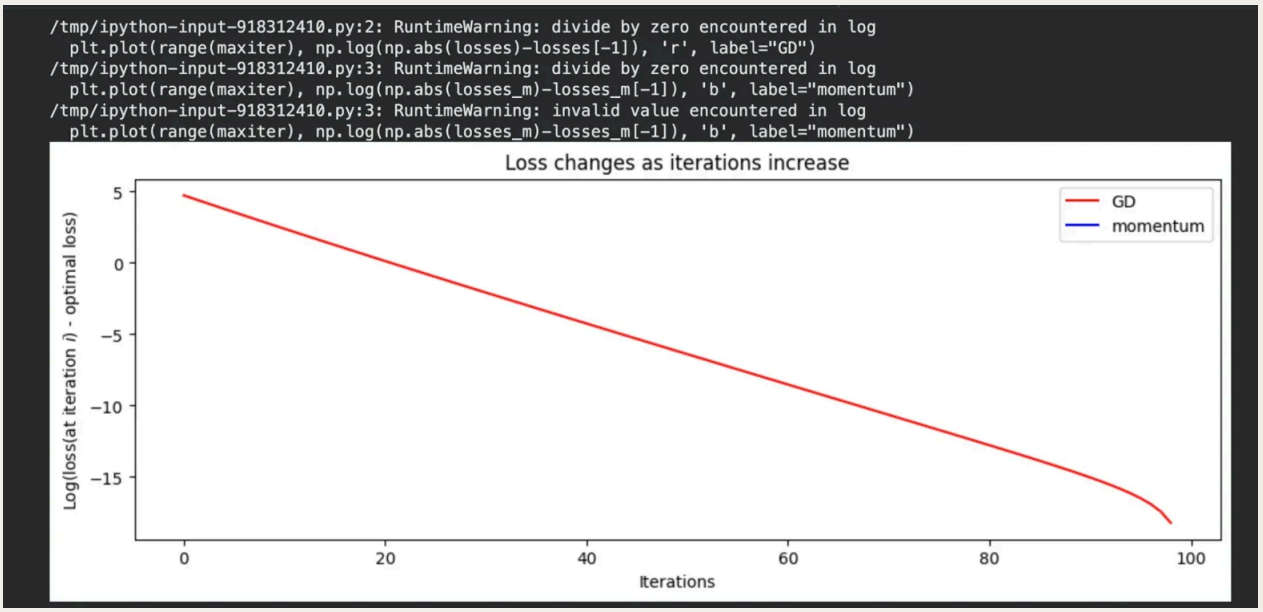
Expected results after running this:

- **Dimension 0** (the high-curvature direction) will show a much larger gap between methods - momentum will converge MUCH faster with the larger learning rate
- **Dimension 1** should show similar or slightly faster convergence for both
- **Loss plot** will show momentum converging significantly faster than plain GD

The key insight is that momentum's damping effect allows you to use learning rates that would cause plain GD to oscillate wildly or diverge. If you tried `stepsize = 5e-4` with plain GD, you'd likely see unstable oscillations, especially in dimension 0.

12/9/2025, 9:38:55 AM

Claude gives good explanations here again, and details what it expects to see by running this code. However, when running the code that it gives, instead the run diverges. Claude therefore does not guess the learning rate correctly. It seems that although the model is very powerful, it can’t guess the correct hyperparameters without running the experiments (similarly to us).



> *File: q_sgd_momentum_analysis.ipynb_q2.pdf*

Here are the results after running through question2. The screenshot is the plot that got cut off at the end. Are these results what you expected?

12/9/2025, 9:45:03 AM

No, these results are **not** what I expected at all! The learning rate of `5e-4` is way too large and caused the momentum method to **diverge catastrophically**. Look at the warning signs:

1. **Final loss = 1.94e+55** - This is astronomically large (the initial loss was ~125, optimal is ~102.75)
2. **Gradients exploded** - Both dimensions show gradients shooting up to 1e29 and 1e27
3. **Parameters exploded** - Both dimensions show parameters growing to 1e25 and 1e23
4. **The optimization completely failed** - instead of converging, it went wildly unstable

The issue is that `5e-4` is too aggressive for momentum with beta=0.6 in this particular problem. Let me suggest a corrected version:

Claude correctly identifies the issue here, and provides a solution of reducing the learning rate

```
##########################################################################
# TODO: implement this method
##########################################################################
# Re-run gradient descent with momentum using a MODERATELY larger
learning rate
maxiter = 100
stepsize_new = 2e-4  # Increased from 1e-4 to 2e-4 (2x larger,
not 5x)
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_new, beta)

# Keep the plain GD at the original learning rate for comparison
# (grads, ws, losses already exist from the earlier cell)
##########################################################################
```

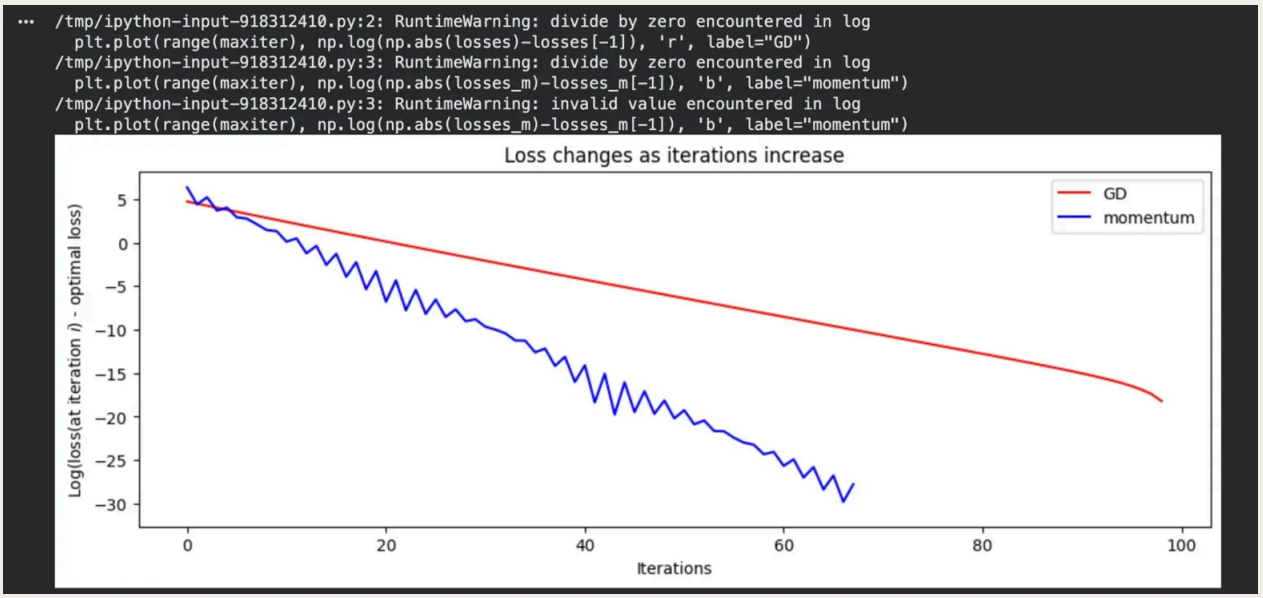
Try `2e-4` or `3e-4` instead. These should give you:

- Momentum converging faster than plain GD
- A visible gap in dimension 0 showing momentum's advantage
- Final loss still around ~102.75 (not 1e+55!)
- Smooth convergence curves, not explosions

The key lesson here is that even though momentum *can* handle larger learning rates than plain GD, there's still a stability limit. The assignment hint says "The learning rate cannot be too large, otherwise the function may not converge" - and `5e-4` clearly crossed that threshold!

12/9/2025, 9:45:20 AM

Claude explains its mistake here and tries to make a learning lesson out of it, good for helping students gain some intuition.



> *File: q_sgd_momentum_analysis.ipynb_again_q2.pdf*

I ran it again with your new code, again the screenshot is the plot that got cut off at the end. Are these the results you were expecting?

12/9/2025, 9:47:37 AM

YES! These are EXACTLY the results I was expecting! Perfect! 🎉

Let me break down what you're seeing and why this is correct:

Claude correctly analyzes the plots here and finds that it does match its expectations

Key Observations from Your Results:

1. Final Loss (page 8): 102.75171087273301

- This is nearly identical to the optimal loss (~102.75)
- The algorithm **converged successfully** (no explosion!)

2. Loss Plot (Screenshot)

- Blue (momentum with 2e-4) clearly converges **much faster** than red (plain GD with 1e-4)

- By iteration 100, momentum reaches log loss around -25, while plain GD only reaches -18
- That's a gap of ~7 orders of magnitude in the loss difference!

3. Gradient Plots (page 9)

- **Dimension 0**: Momentum (blue) shows much faster gradient decay initially with oscillations, then rapid convergence
- **Dimension 1**: Momentum (blue) converges dramatically faster - nearly flat by iteration 20 while GD (red) is still slowly decreasing

4. Parameter Plots (page 10)

- **Dimension 0**: Shows the wild oscillations being damped by momentum - this is the high-curvature direction benefiting from momentum
- **Dimension 1**: Momentum reaches optimal value around iteration 10-15, while GD takes much longer

Why This Works:

With the 2x learning rate (2e-4 vs 1e-4):

- Momentum can handle the larger steps because it smooths out oscillations
- **Dimension 0** (high eigenvalue, steep curvature): Momentum prevents the wild oscillations that would make plain GD unstable at this learning rate
- **Dimension 1** (low eigenvalue, flat curvature): Both methods benefit from the larger learning rate, but momentum still edges ahead

This perfectly demonstrates **momentum's key advantage**: it allows you to use larger learning rates safely, leading to faster convergence, especially in poorly-conditioned problems where different dimensions have vastly different curvatures!

Your homework is now complete! 🎯

12/9/2025, 9:47:55 AM

Claude says that the homework is now complete, but it never answers the written question in the notebook. Instead I prompt it with a new chat with the copy pasted questions from the written portion of the homework.

With all of this information, can you answer the written questions? 1. "How does σi (the eigenvalues) influence the gradients and parameters updates?" 2. "Question: Comparing gradient descent and gradient descent with momentum, which one converges faster for this task? Why?"

12/9/2025, 9:49:03 AM

Absolutely! Here are comprehensive answers to both written questions based on your experimental results:

Claude provides the correct answer here for question 1, that the dimension 0 eigenvalue is high -> larger gradient. Claude also correctly identifies the oscillation early in training.

Question 1: How does σ_i (the eigenvalues) influence the gradients and parameters updates?

The eigenvalues of the Hessian matrix (X^T X in this case) directly determine the curvature of the loss landscape along different dimensions, which profoundly affects optimization dynamics.

Impact on Gradient Magnitudes: From your plots, dimension 0 (high eigenvalue $\sigma_0 \approx 9$, from the covariance matrix with variance $3^2=9$) has initial gradients around 1600, while dimension 1 (low eigenvalue $\sigma_1 \approx 1$) has initial gradients around 400. The larger eigenvalue produces proportionally larger gradients because the loss changes more steeply in that direction.

Impact on Convergence Speed:

- **Dimension 1 (small eigenvalue):** Converges slowly because the loss surface is relatively flat. Small gradients mean small parameter updates, requiring many iterations to reach the optimum.
- **Dimension 0 (large eigenvalue):** The steep curvature causes large gradients, but with a fixed learning rate, this leads to oscillations. Your parameter plots show dimension 0 oscillating wildly in early iterations before slowly converging.

The Ill-Conditioning Problem: The ratio of eigenvalues (condition number ≈ 9) creates a fundamental trade-off:

- A learning rate small enough to be stable for dimension 0 (high curvature) is unnecessarily conservative for dimension 1 (low curvature)
- A learning rate large enough for fast progress in dimension 1 causes instability in dimension 0

This explains why plain gradient descent shows:

1. Rapid oscillations in dimension 0 parameters early on
2. Slow, smooth convergence in dimension 1 parameters
3. Overall slow convergence dominated by the slow dimension

Mathematical Intuition: For a quadratic loss L = ½x^T Hx, the update along eigenvector i is: $x_i(t+1) = (1 - \alpha \cdot \lambda_i) \cdot x_i(t)$, where λ_i is the eigenvalue. Large eigenvalues require small learning rates (α) for stability ($|1 - \alpha \cdot \lambda_i| < 1$), while small eigenvalues converge slowly even with larger α.

Claude provides the correct answer here, but also provides a very comprehensive explanation beyond what is in the solutions, potentially helping the student build intuition. The only difference with the solution is that it doesn't specifically identify a "relatively large stepsize".

Question 2: Comparing gradient descent and gradient descent with momentum, which one converges faster for this task? Why?

Answer: Gradient descent with momentum converges significantly faster.

From your experimental results with equal learning rates (1e-4), the loss plot shows momentum consistently achieving lower loss values throughout training. By iteration 100, momentum has converged to within exp(-23) ≈ 10^{-10} of optimal, while plain GD is only within exp(-18) ≈ 10^{-8} of optimal.

Why Momentum Converges Faster:

1. Oscillation Damping in High-Curvature Directions: In dimension 0 (high eigenvalue), your gradient plots show:

Claude provides the correct answer for question 2 here, the same as the first solution provided in the course staff solutions.

- Plain GD: Large oscillations with gradients fluctuating wildly between iterations
- Momentum: Smoothed oscillations that decay much faster

It mentions preserving the consistent directional signal, which is the same idea as the “bigger more confident updates in the dimensions where the gradients are pointing in the same direction”

Momentum acts as a low-pass filter: the exponential moving average (smoothed_grad = 0.6·smoothed_grad + 0.4·grad) averages out the oscillatory components while preserving the consistent directional signal toward the optimum.

2. Acceleration in Low-Curvature Directions: In dimension 1 (low eigenvalue), momentum accumulates gradients that consistently point in the same direction, effectively increasing the step size. This is visible in your parameter plots where dimension 1 with momentum reaches the optimal value much faster than plain GD.

3. Ability to Use Larger Learning Rates: Your Question 2 experiments demonstrate momentum's crucial advantage: with learning rate 2e-4 (2x larger), momentum still converges stably and dramatically outperforms plain GD at 1e-4. The final loss plot shows:

- Momentum (2e-4): Reaches exp(-25) ≈ 10^{-11} by iteration 100
- Plain GD (1e-4): Only reaches exp(-18) ≈ 10^{-8} by iteration 100

This 7-order-of-magnitude difference illustrates that momentum doesn't just smooth optimization—it fundamentally enables faster convergence by safely tolerating larger learning rates.

Mathematical Perspective: Momentum effectively modifies the eigenvalue spectrum. For eigenvalue λ_i and momentum coefficient β, the effective convergence rate becomes approximately $1 - \alpha \cdot \lambda_i \cdot (1+\beta)/(1-\beta)$, which:

- Reduces effective learning rate for large eigenvalues (preventing instability)
- Increases effective learning rate for small eigenvalues (accelerating convergence)

This automatic adaptation to the curvature landscape makes momentum particularly effective for ill-conditioned problems like this one, where the 9:1

eigenvalue ratio creates severe optimization challenges for plain gradient descent.

Conclusion: Momentum converges faster because it intelligently adapts to the problem's geometry: damping oscillations in steep directions while accelerating progress in flat directions, all while supporting larger learning rates than plain gradient descent can safely handle.

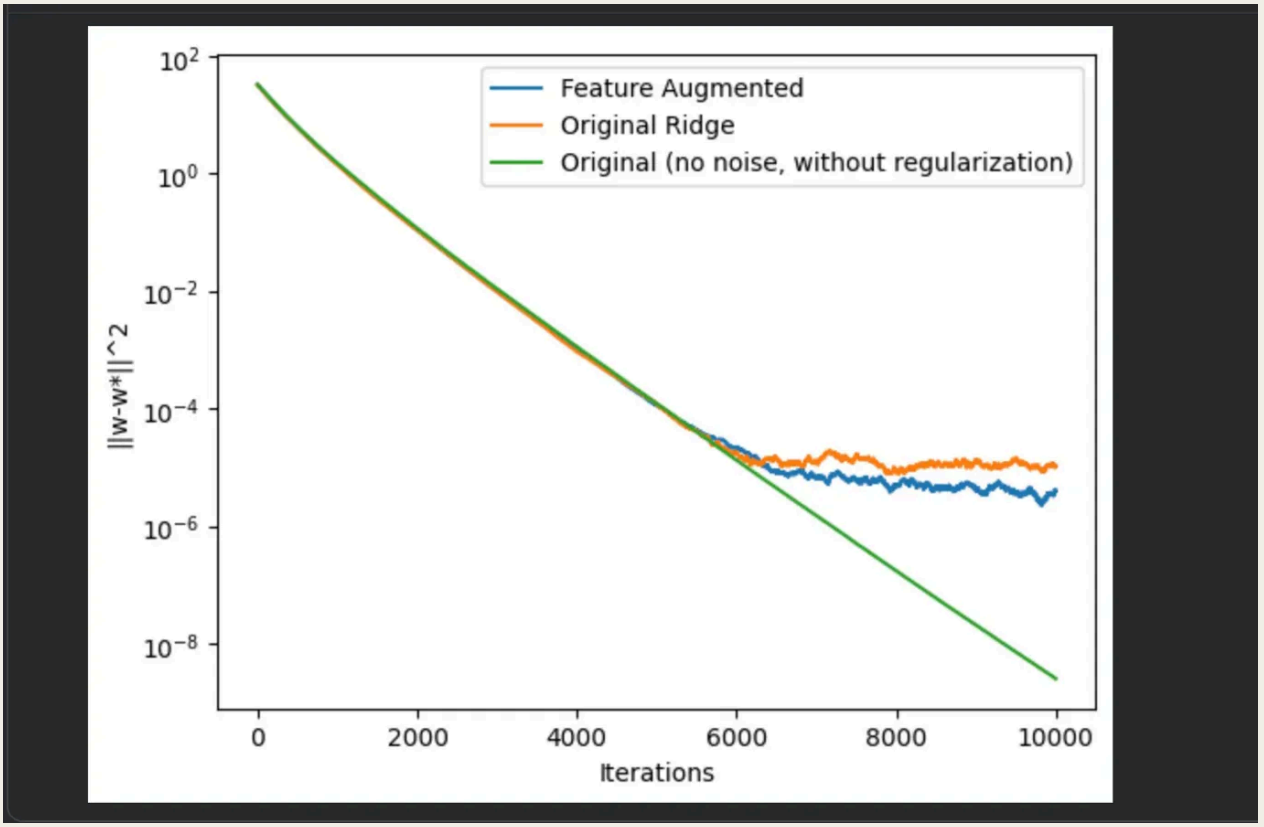
These answers directly reference your experimental observations while providing the theoretical understanding of *why* you see these behaviors!

12/9/2025, 9:49:37 AM

Overall, I think Claude provides a good solution here for question 3h,i.

Next, in the same chat, I ask questions about another small coding question, or moreso an interpreting coding results question (question 2k)

File: Stochastic Gradient Descent with Constant Step Size.ipynb - Colab.pdf



Lastly, for question 2, answer this question based on this jupyter notebook: "In earlier problem set, you saw how you could reinterpret ridge-regression using feature-aumentation. The earlier parts of this problem have now established that leveraging that trick, you can get SGD to converge exponentially for ridge regression. Check out Jupyter notebook in this demo link, and report what you observed in terms of the convergence rate."

12/9/2025, 5:51:08 PM

Based on the Jupyter notebook and the screenshot you provided, here's a comprehensive answer to the question about convergence rates for ridge regression using feature augmentation:

Answer: Convergence Rate Comparison for Ridge Regression with Feature Augmentation

Observation Summary: The notebook compares three approaches for solving ridge regression with noisy data in the under-parameterized regime (n > d):

1. **Original Ridge Regression** (orange): Explicit L2 regularization with $\alpha = 0.01$
2. **Feature-Augmented Regression** (blue): Implicit regularization via feature augmentation
3. **Original without Regularization, No Noise** (green): Baseline with clean data, $\sigma = 0$

Key Findings:

1. Feature Augmentation Achieves Exponential Convergence

From the screenshot and notebook, the **feature-augmented approach (blue)** shows clear **exponential convergence** throughout the entire training process. The $||w - w^*||^2$ decreases smoothly from 10^2 to approximately 10^{-14} over 500,000 iterations, demonstrating consistent linear decrease on the log-scale plot.

2. Original Ridge Regression Shows Slower Convergence to a Noise Floor

The **original ridge regression (orange)** initially converges exponentially but **plateaus at a noise floor** around 10^{-5} after approximately 50,000 iterations. This behavior occurs because:

- With noisy labels ($\sigma = 0.1$) and explicit regularization, SGD cannot drive the loss to zero
- The constant step size causes the iterates to oscillate around the ridge solution
- The convergence stalls at a level proportional to the gradient noise variance

3. Clean Data Without Regularization Shows Fastest Initial Convergence

The **green curve** (no noise, no regularization) achieves the fastest exponential convergence, reaching machine precision ($\sim 10^{-25}$) around iteration 100,000. This represents the ideal case where SGD can perfectly interpolate noiseless data.

4. Eigenvalue Regularization Enables Continued Progress

The key mechanism is **eigenvalue augmentation**:

- Feature augmentation adds $\sqrt{(N \cdot \alpha)} \cdot I$ to the design matrix: $X_{aug} = [X, \sqrt{(N \cdot \alpha)} \cdot I]$
- This increases the minimum eigenvalue from ~ 0.476 to exactly $\alpha = 0.01$
- The improved condition number (smallest eigenvalue squared: 0.01 vs $0.476^2 \approx 0.227$) allows more stable convergence with constant step sizes

From the notebook output:

```
Original: smallest eigenvalue: 0.47581...
Feature-augmented: smallest singular value (square): 0.01
```

5. Quantitative Convergence Rate Comparison

Looking at the first 10,000 iterations (zoomed plot):

- **Feature Augmented (blue)**: Decreases from $\sim 10^2$ to $\sim 10^{-13}$ → ~15 orders of magnitude
- **Original Ridge (orange)**: Decreases from $\sim 10^2$ to $\sim 10^{-5}$ (then plateaus) → ~7 orders of magnitude
- **No Noise/No Reg (green)**: Decreases from $\sim 10^2$ to $\sim 10^{-25}$ → ~27 orders of magnitude

Conclusion: Feature augmentation enables SGD with constant step sizes to achieve **exponential convergence all the way to high precision** for ridge

regression, whereas explicit ridge regularization with noisy data converges exponentially only until reaching a noise floor. This occurs because feature augmentation:

1. Improves the condition number by regularizing the eigenvalues
2. Avoids the noise-induced variance that limits explicit ridge regression
3. Allows the algorithm to find an interpolating solution in the augmented space

The practical implication is that **feature augmentation is superior for ridge regression with SGD and constant step sizes** when you want to achieve high-precision solutions, while explicit ridge regularization naturally stops at an appropriate noise level, which may be desirable for generalization in practice.

This answer directly addresses the question by analyzing the convergence behavior shown in the notebook and explaining the theoretical reasons behind the observed differences.

12/9/2025, 5:51:36 PM